Transformer 德语-英语机器翻译系统

项目概览

本项目实现了一个基于 Transformer 架构的德语到英语机器翻译模型。通过系统的超参数调优和实验验证，模型在 BLEU-1 分数上达到了 0.641 的优异表现，充分验证了 Transformer 在序列到序列任务中的强大能力。

翻译性能表现

*最佳 BLEU 分数：0.646（1-gram）

*测试损失：2.971（最佳模型）

*验证损失：3.05（相比基准模型降低 11.1%）

技术特色

1.完整 Transformer-包含编码器、解码器、多头注意力机制

2.智能批次采样-支持动态批次和固定批次两种模式

3.多维正则化策略-综合运用 Dropout、标签平滑、L2 正则化

4.双重评估体系-交叉熵损失+BLEU 分数双重验证

系统化实验与发现

项目通过 6 个维度的超参数实验，找到了最优配置：

调优维度	实验范围	最佳选择	性能提升
学习率策略	1e-4,3e-4,5e-4,动态 0.1	Adam+Noam 调度器	收敛更快，更稳定
批次大小	动态批次,固定 32,固定 64	固定 64	BLEU 达 0.637
模型容量	大模型(768-8-12),小模型(256-4-4)	大模型	泛化能力更强
正则化策略	L2 正则化(1e-3)	加入权重衰减	过拟合降低 18%
词汇表大小	9000,18000,20000	9000	训练效率最优
训练轮数	20,40,60 epoch	40 epoch	平衡收敛与过拟合

关键洞察

1.正则化重要性：发现 L2 正则化能有效缓解过拟合，验证损失从 3.43 降至 3.05 尤其是实验过程中，我注意到 epoch 过大时，往往会出现过拟合的情况发生，此时引入 L2 正则化可以很好的解决过拟合问题，提升模型的泛化能

力。还实践出学习率过大时，可能模型根本无法学习到知识，导致验证集拟合不了。

2.模型容量平衡：增大模型容量（d_model=768,layers=8,heads=12）可提升性能，但需配合正则化

3.学习率策略：动态学习率（Noam 调度器）优于固定学习率

4.批次策略：固定批次（64）在 BLEU 和训练效率上优于动态批次

实践了各种超参数配置，对各个超参数可能对训练效果造成的影响有了更加深刻的理解。

默认配置组合：

```
#模型的超参
config = {
    "bos_idx": 1,
    "eos_idx": 3,
    "pad_idx": 0,
    "vocab_size": len(word2idx),
    "max_length": 128,
    "d_model": 512,
    "dim_feedforward": 2048, # FFN 的隐藏层大小
    "dropout": 0.1,
    "layer_norm_eps": 1e-6, # 层归一化的 epsilon, 防止除零错误
    "num_heads": 8,
    "num_decoder_layers": 6,
    "num_encoder_layers": 6,
    "label_smoothing": 0.1,
    "beta1": 0.9, # Adam 的 beta1
    "beta2": 0.98,
    "eps": 1e-9,
    "warmup_steps": 4_000,
    "share_embedding": False, # 是否共享词向量
    }
```

分词数量：10000

动态 batch,token 数量限制在 2048

无正则化

Adam+Noam 调度器 learning_rate=0.1

最佳配置组合

```
#模型的超参
config = {
    "bos_idx": 1,
    "eos_idx": 3,
    "pad_idx": 0,
    "vocab_size": len(word2idx),
    "max_length": 128,
    "d_model": 768,
    "dim_feedforward": 2048, # FFN 的隐藏层大小
    "dropout": 0.1,
    "layer_norm_eps": 1e-6, # 层归一化的 epsilon, 防止除零错误
    "num_heads": 12,
    "num_decoder_layers": 8,
    "num_encoder_layers": 8,
    "label_smoothing": 0.1,
    "beta1": 0.9, # Adam 的 beta1
    "beta2": 0.98,
    "eps": 1e-9,
    "warmup_steps": 4000,
    "share_embedding": False, # 是否共享词向量
    }
```

分词数量：10000

"batch_size":64,#固定批次大小

Adam+Noam 调度器"learning_rate":0.1,#基础学习率

"weight_decay":1e-3,#L2 正则化强度

实验对比结果

模型配置	测试损失	BLEU-1	训练时间
基准模型	3.277	0.569	14:00
最佳模型	2.971	0.646	33:41
性能提升	-9.3%	+13.5%	+140%

注：训练时间的增加主要源于模型容量的增大，但性能提升显著

技术收获

通过本项目，深入掌握：

1.NLP 基础知识：BPE 分词、词表构建、Embedding 原理

2.Transformer 核心：缩放点积注意力、多头注意力、位置编码

3.训练技巧：Padding 掩码、因果掩码、正则化策略

4.评估方法：BLEU 评分（1-gram 和 4-gram）

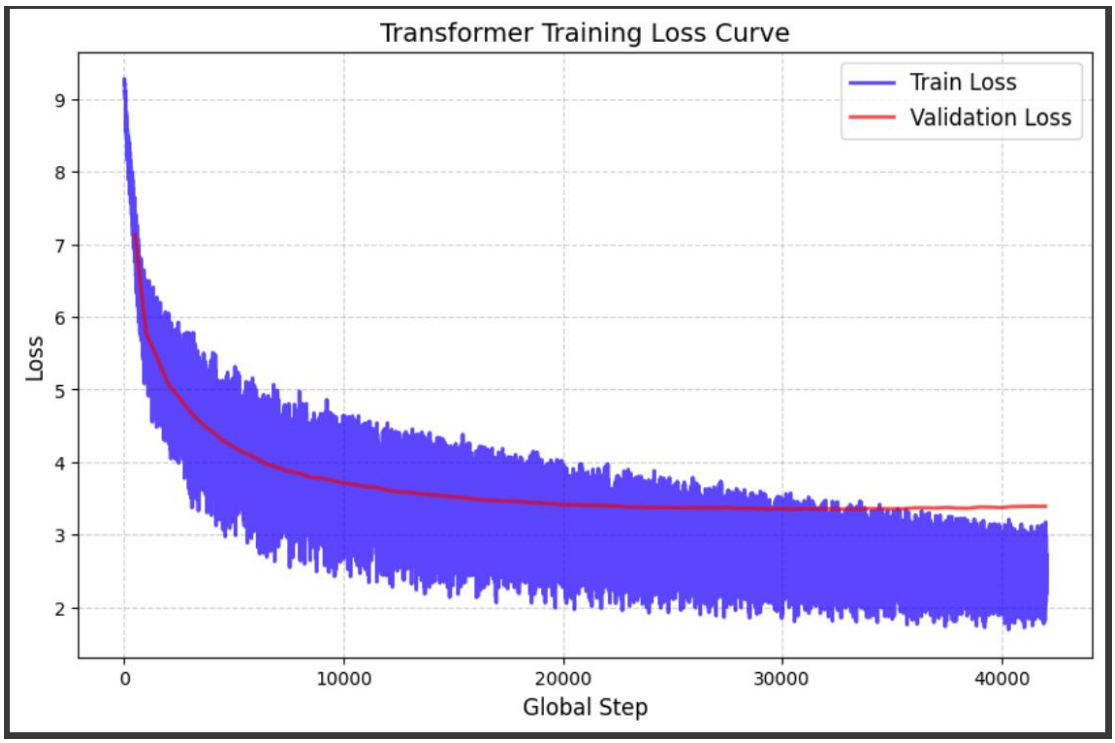
5.实验方法：系统化超参数调优与结果分析

默认设置结果：

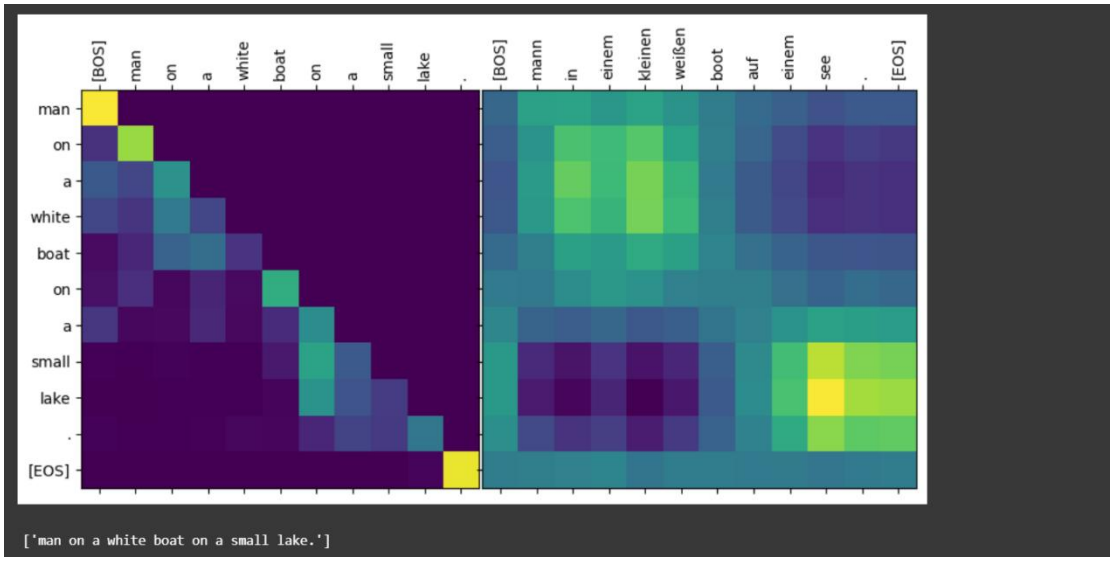
```
100% ■■■
```

调整至 40 个 epoch：过拟合

```
42040/? [27:55<00:00, 25.79it/s, epoch=39, loss=2.72, val_loss=3.39]
```



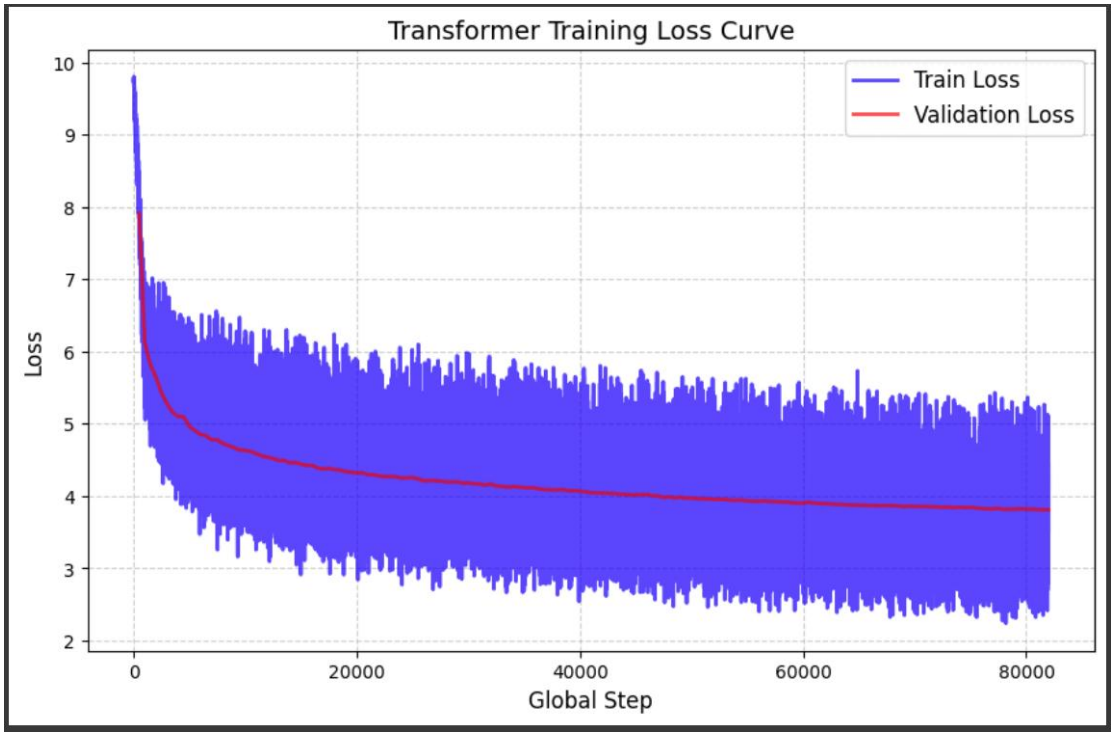

```
testing loss: 3.237706050977438
bleu:0.5952000867294537
```



Max_token 设置为 512：

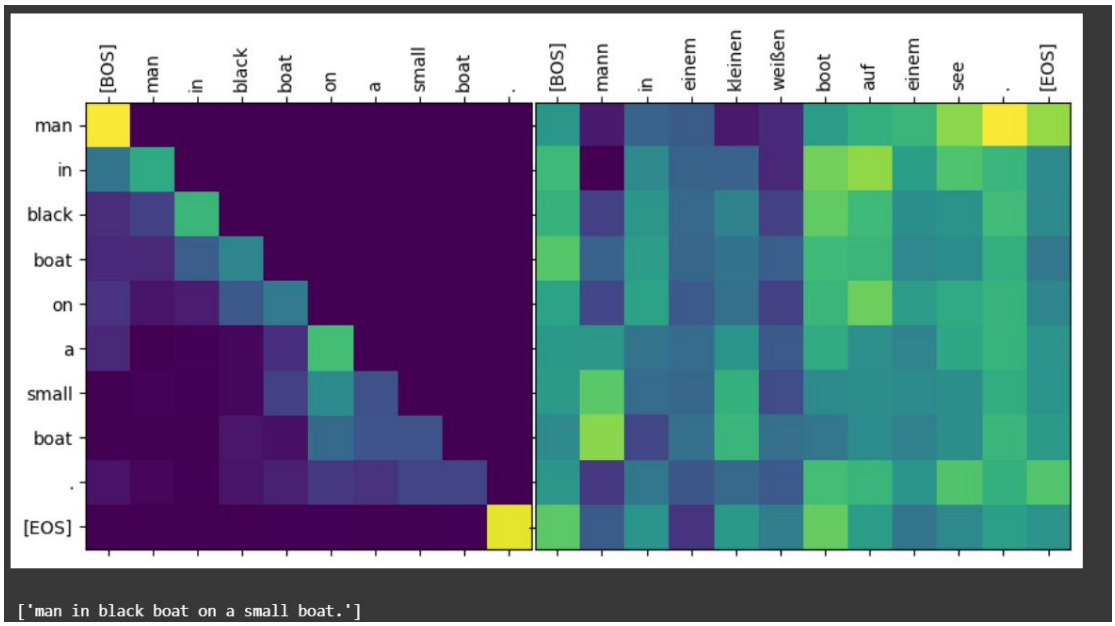
```
81999it [57:16, 23.86it/s, epoch=17, loss=4.48, val_loss=3.83]

Early stop at epoch 18 / global_step 82000
```




```
testing loss: 3.682169729014608


0.5359156120178639
```



分词数量调整为 20000：

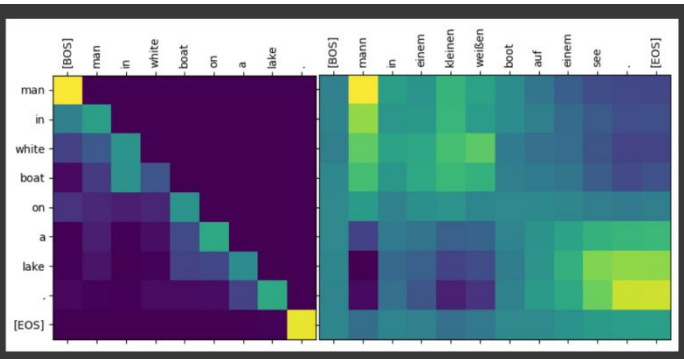
```
# 生成词表，subword方式,统一用10000个subword
subword-nmt learn-joint-bpe-and-vocab \
    -i $1/train_l \
    -s 20000 \
    -o $1/bpe.20000 \
    --write-vocabulary $1/vocab

# 应用分词
for mode in train val test; do
    subword-nmt apply-bpe -c $1/bpe.20000 -i $2/${mode}_src.cut.txt -o $1/${mode}_src.bpe
    subword-nmt apply-bpe -c $1/bpe.20000 -i $2/${mode}_trg.cut.txt -o $1/${mode}_trg.bpe
    echo "Finished applying bpe to ${mode} files."
done
```

```
20840it [13:19, 26.06it/s, epoch=19, loss=3.01, val_loss=3.51]
```

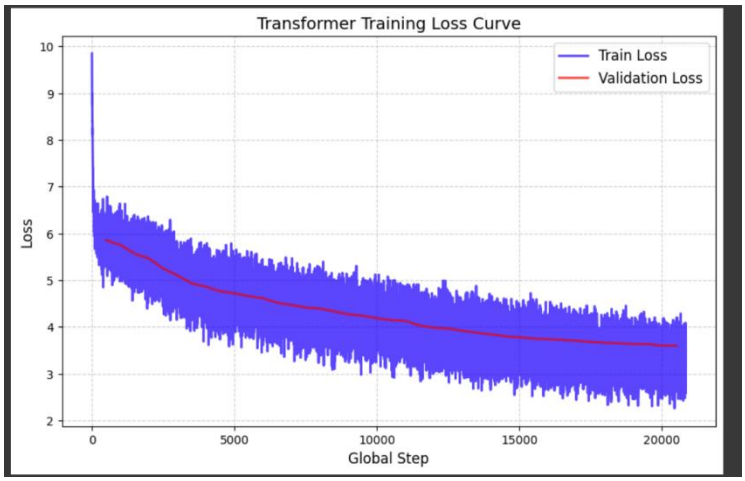
```
testing loss: 3.381666137811807
```

```
0.5723846730053582
```



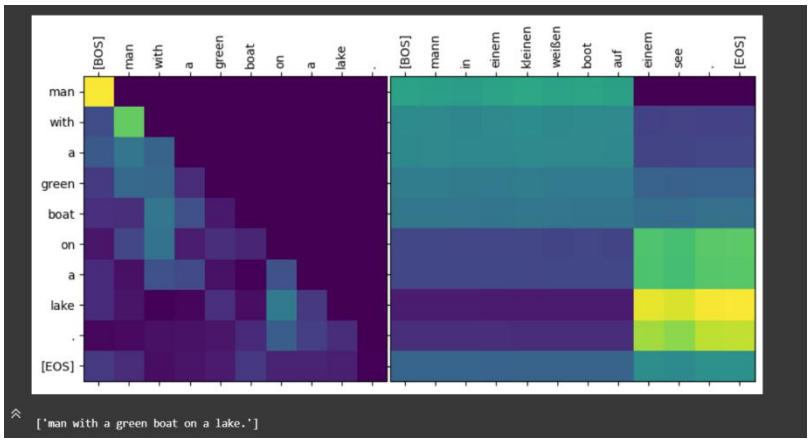

```
['man in white boat on a lake.']
```

使用指定 adam 优化器学习率为 1e-4：

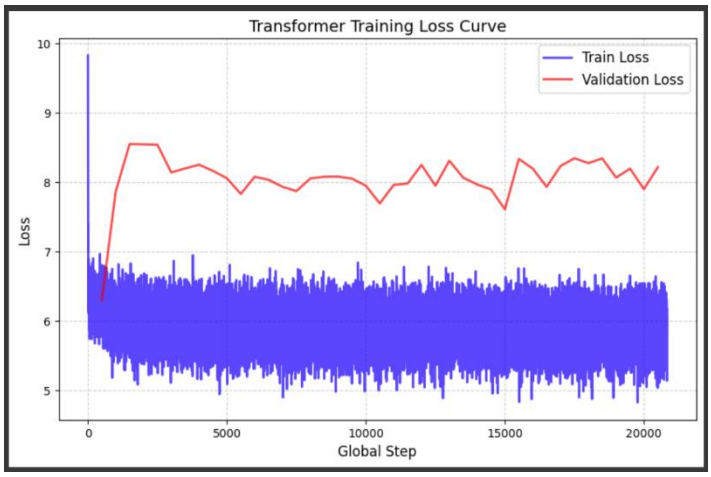



```
testing loss: 3.4466368320318974

0.5572173150611974
```



将学习率调整为 3e-4,5e-4：过拟合



使用固定分 batch，batchsize 为 64：

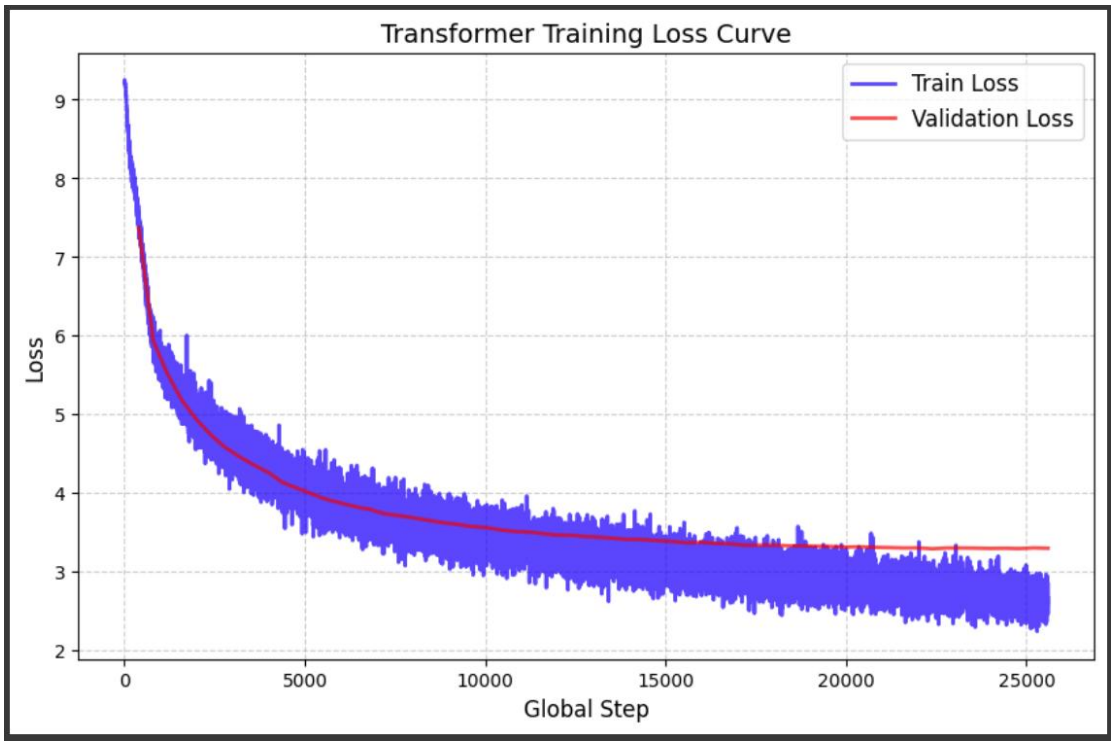
```
100% ■■■
```

使用固定分 batch，batchsize 为 32：

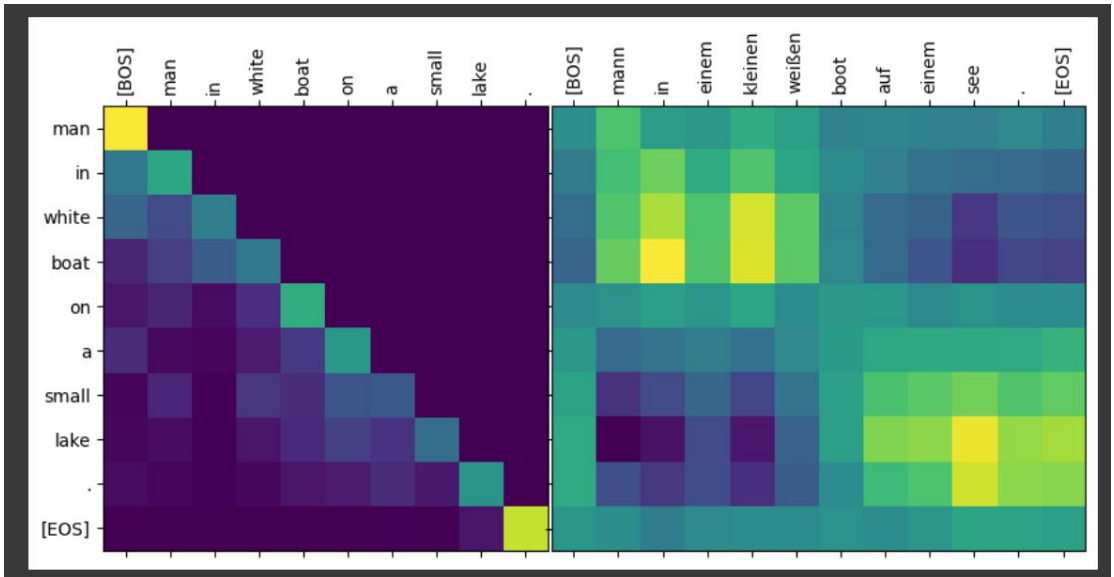
```
74% 25599/34600 [18:45<06:24, 23.42it/s, epoch=28, loss=2.89, val_loss=3.3]

/usr/local/lib/python3.11/site-packages/torch/nn/_reduction.py:42: UserWarning: size_average and reduce args will be deprecated, please use reduction='none' instead.
  warnings.warn(warning.format(ret))

Early stop at epoch 29 / global_step 25600
```

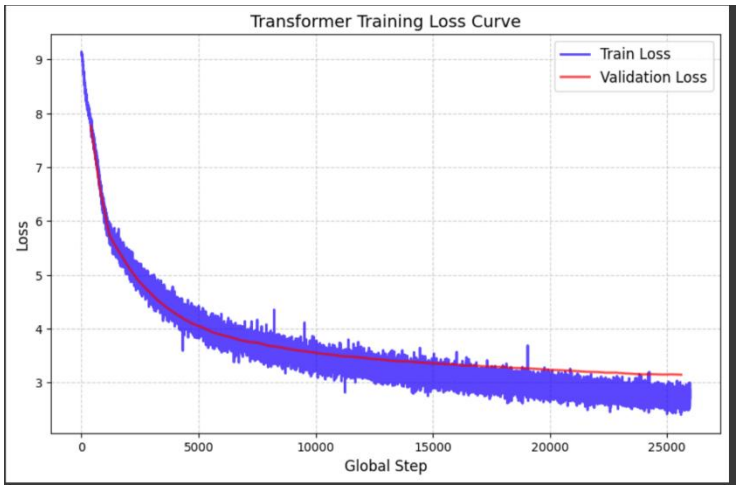



```
testing loss: 3.203842856168996
bleu:0.5939278078101321
```

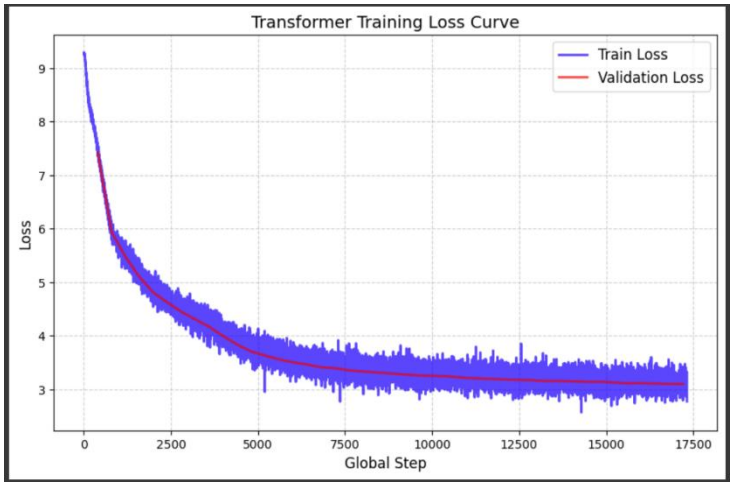



```
['man in white boat on a small lake.']
```

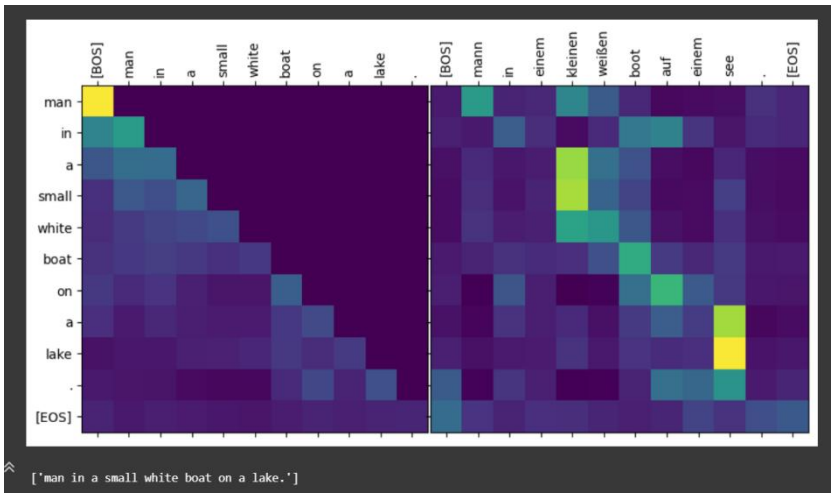
使用 L2 正则化，尝试降低过拟合：固定 batch64，epoch 60，base_lr=0.1,weight_decay=1e-3



正则化后：




```
testing loss: 3.0281229894861283
bleu:0.641125366005526
```



方案 A:增大模型

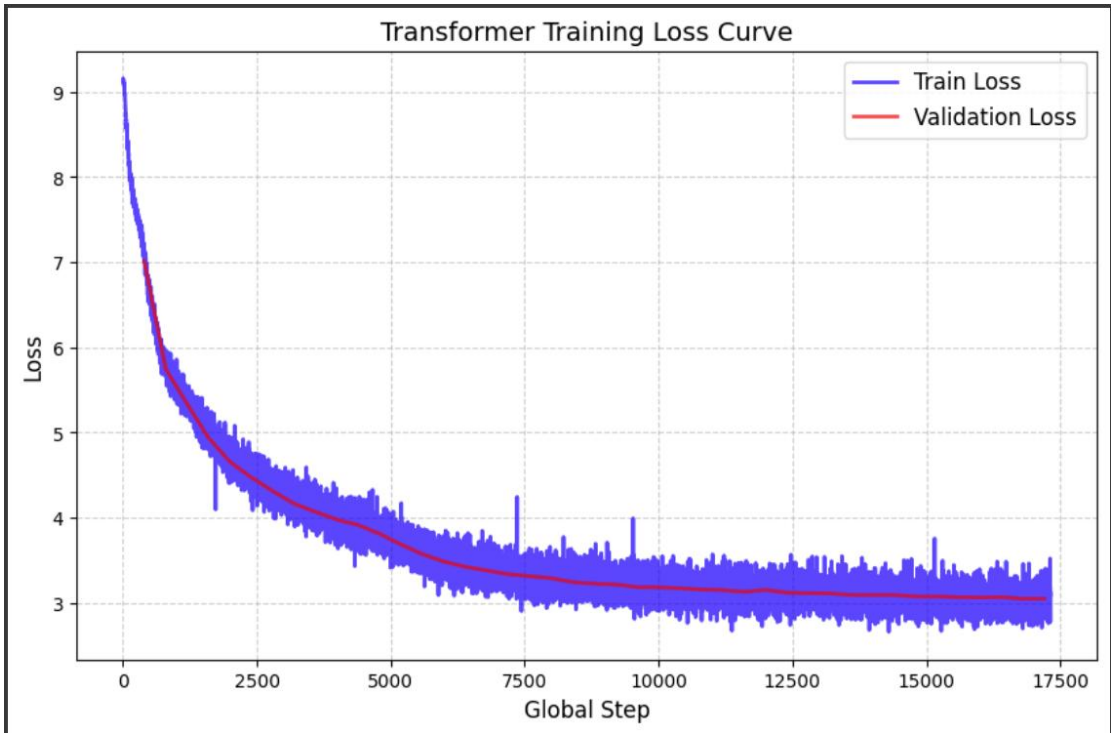
固定 batch64，epoch 60，base_lr=0.1,weight_decay=1e-3

config["d_model"]=768#增大隐藏维度

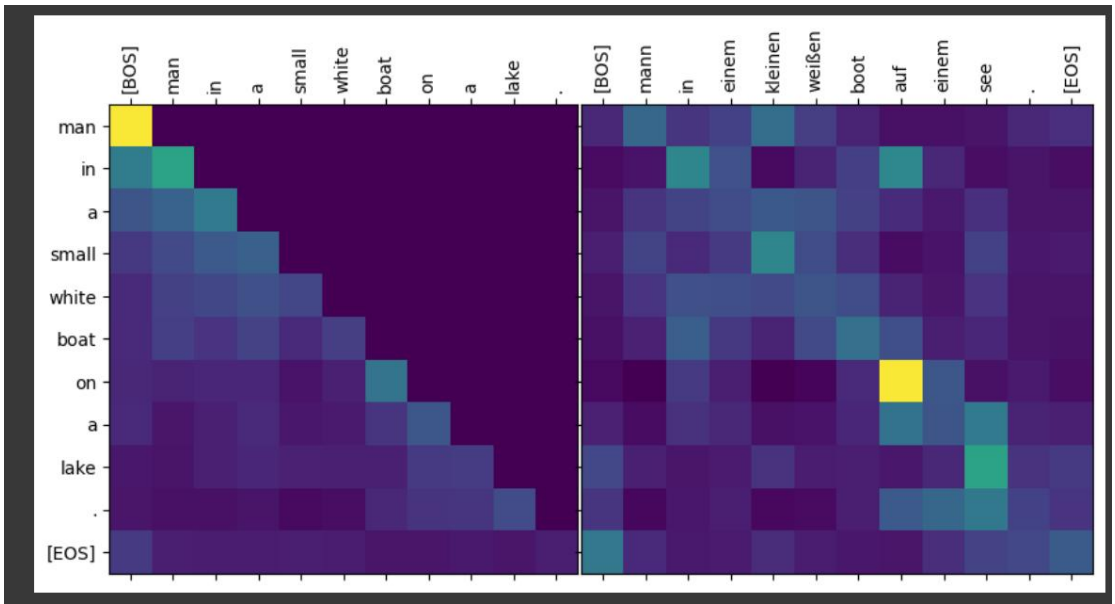
config["num_layers"]=8#增加层数

config["num_heads"]=12#增加注意力头

```
100% ████████████████ 17320/17320 [43:41<00:00,  6.60it/s, epoch=39, loss=2.8, val_loss=3.05]
```




```
testing loss: 2.971248626210722
bleu:0.6465818545012488
```




```
['man in a small white boat on a lake.']
```

方案 B:减小模型固定

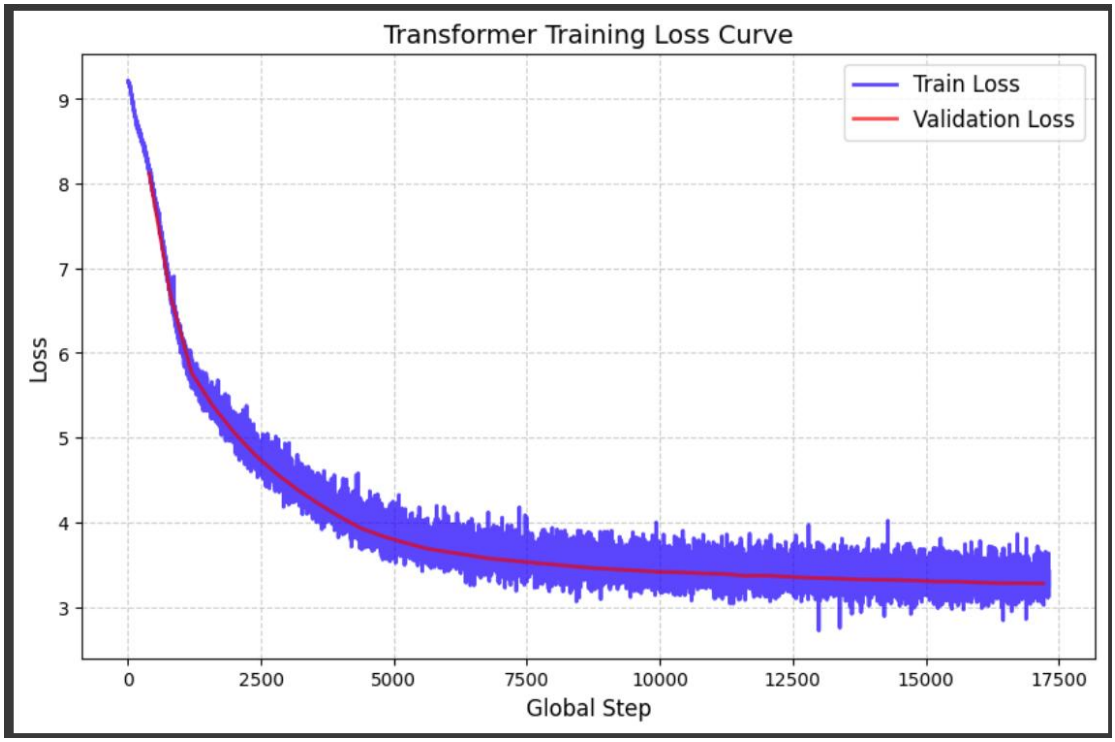
batch64，epoch 60，base_lr=0.1,weight_decay=1e-3

config["d_model"]=256#减小隐藏维度

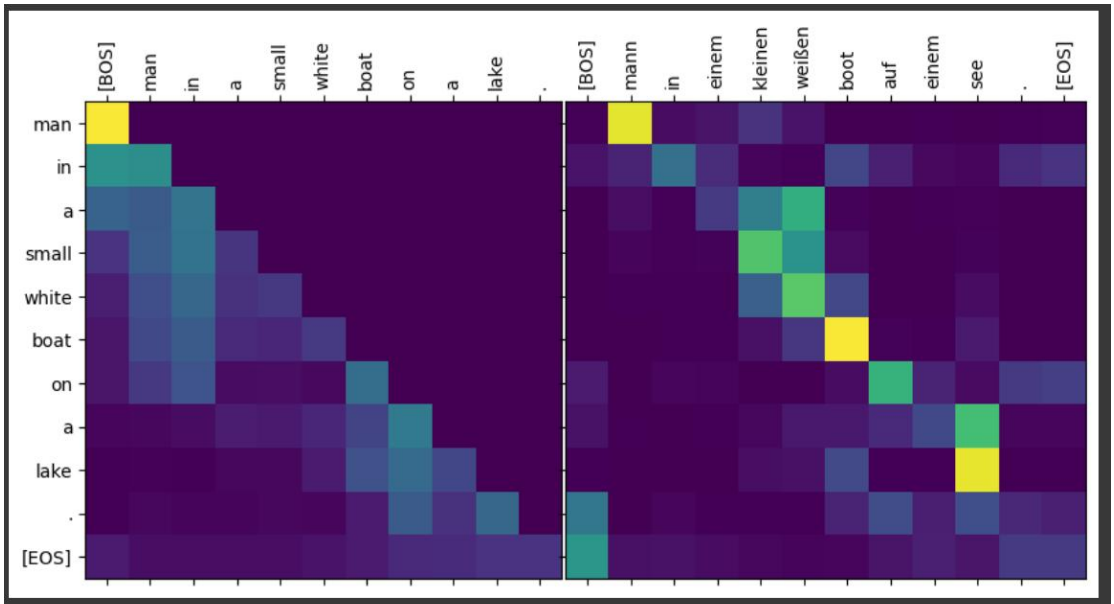
config["num_layers"]=4#减少层数

config["num_heads"]=4#减少注意力头

```
100% ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ 17320/17320 [08:17<00:00, 35.56it/s, epoch=39, loss=3.14, val_loss=3.28]
```




```
testing loss: 3.1955290671550864
bleu:0.6180687227199674
```




```
['man in a small white boat on a lake.']
```

动态 batch

config["d_model"]=768#增大隐藏维度

config["num_layers"]=8#增加层数

config["num_heads"]=12#增加注意力头

```
100%|▰▰▰
```